

Weighted Graph-Structured Lambda Theories

Stochastic and quantum execution for MeTTaIL, with recurrent
neural networks as a worked example

A F1R3FLY.io working note

August 2026

Abstract

We equip the rewrite rules of a finitely presentable graph-structured lambda theory (GSLT), as presented in MeTTaIL, with *weight maps*: finite maps whose keys are spatial-behavioural formulae refining the left-hand side of a rule and whose values are weights. A weight map instance is part of a state specification, so a rewrite may update it; weights are therefore dynamic. We develop the construction in three movements. First, we impose a *partition discipline* on the key set — the refinements of a rule’s left-hand side must be pairwise exclusive and jointly exhaustive — and show that this single requirement is what makes the weighting a total, single-valued classification of redexes, forces the generated logic’s target specification to be Boolean, and, in the complex-valued case, makes the refinement classes a projective measurement. Second, with weights in $\mathbb{R}_{\geq 0}$ we define propensity as $\mathbf{a} = w(\varphi) \cdot \sum_k g(k)$ over classified, funded redex positions and obtain a Gillespie simulator; we prove that the construction restricted to the π -calculus, static maps, and channel-identity keys is exactly the Stochastic Pi Machine, and that dynamic maps take it strictly beyond. Third, with weights in $\mathbb{C}$ we read each refinement entry as a Lindblad jump operator, obtain a quantum continuous-time Markov chain in the sense of Xu, Mei, Guan and Yu, and show that the Gillespie algorithm is the quantum-jump unravelling at zero Hamiltonian — which is the precise sense in which the quantum simulator is *Gillespie-inspired*. The worked example is a recurrent neural network in a weighted rho theory: neurons are persistent for-comprehensions, synaptic efficacies live in the weight map rather than in the payload, thresholds are join structure, the logistic response is emergent from the chain rather than written into a term, and Hebbian plasticity is exactly the refinement entries’ update functions — so that learning and inference are the same transition relation, differing only in which component of the state they touch.

Contents

1	Introduction	2
2	Background, compressed	4
2.1	GSLTs and their presentations	4
2.2	Locations	5
2.3	The generated logic	5
2.4	Ideal logic, checkable fragment	6
3	Refining a left-hand side	6
3.1	The keys	6
3.2	What the partition buys	7
3.3	The partition forces a Boolean target specification	7
3.4	Locality	8

4	Weighted rewrite systems	8
4.1	The weight codomain	8
4.2	Weight maps and augmented rules	8
4.3	Why context rules carry a factor and not a propensity	9
4.4	Dynamics	9
5	Propensity	10
6	Real weights: the Gillespie construction	11
6.1	The simulator	11
6.2	SPiM is the static, name-keyed, ungeometric restriction	12
6.3	Finiteness	13
7	Complex weights: a quantum continuous-time Markov chain	14
7.1	The space	14
7.2	Jumps from refinements, coherence from equations	14
7.3	Interference	15
7.4	Gillespie as a degenerate quantum jump	15
7.5	The generated logic supplies the atomic propositions	16
8	Worked example: a recurrent neural network	17
8.1	The design decision that makes the example an example	17
8.2	Neurons are persistent for-comprehensions	17
8.3	The keys, and why they form a partition	18
8.4	Threshold is join structure	18
8.5	The weighted sum, and where the squashing function went	19
8.6	Boundedness	20
8.7	Plasticity is the update function	20
8.8	Spike-timing dependence needs a wider update signature	21
8.9	Uniformity: one theory for every network	21
8.10	The quantum network	22
8.11	What this example does not show	22
9	Implementation	22
9.1	Surface syntax	22
9.2	Delta against the prototype	23
9.3	Incremental propensity	24
10	Weights and costs	24
11	Open threads	25

1 Introduction

A rewrite rule’s left-hand side is a type. Terms inhabit it; the rule fires at its inhabitants. This note is about what happens when one refines that type and attaches a number to each refinement.

The move is small to state and consequential to develop. A GSLT is a triple (Σ, E, R) : a signature which is in general a lambda theory, a set of equations imposing a structural congruence, and an adjoined graph of rewrite rules [16]. Given such a presentation, the OSLF family of algorithms generates a logic from it, with one structural connective per term

former of Σ and one context-labelled modality per rule of R together with a choice of redex position [16, 1]. The formulae of that generated logic are exactly the vocabulary in which one can refine a rule’s left-hand side — and they are generated, not designed, so the refinements come for free with the theory. To weight a rule is to give a finite map

$$\varphi_1 \mapsto v_1, \quad \dots, \quad \varphi_m \mapsto v_m$$

from such refinements to weights. A term’s redexes are then classified by which refinement they satisfy, and the weights turn the set of enabled redexes into a distribution.

Two things distinguish this from the existing literature on stochastic process calculi.

Keys are formulae, not names. In Priami’s stochastic π -calculus [17] and in Phillips and Cardelli’s Stochastic Pi Machine (SPiM) [19, 20], a rate is attached to a *channel*. That is one refinement among many: “this redex is a communication on n ” is a particular formula in the generated logic, and there is no reason the modeller should be restricted to it. Once keys are formulae, a rate may depend on the shape of the payload, on the structure of the surrounding term as seen through the separating conjunction, on the namespace the channel belongs to, or on any Boolean combination of these. Channel identity is recovered as the degenerate case (§6.2).

Maps are state, not declaration. A weight map instance is part of a state specification. A rule’s refinement entries therefore carry not only a weight but an *update function* on the whole map, applied when the rule fires. Rates change during execution. SPiM does not permit this — its rates are fixed at declaration — and the restriction is not incidental to its design but constitutive of it, since a static rate function is what makes the underlying object a time-homogeneous CTMC over terms alone. We recover Markovianity by a different route: the weight map is *in* the state, so the chain is over configurations (P, w) rather than over terms (§6). This is the technical content of “the map may be updated,” and it is what the worked example of §8 needs, because plasticity is nothing other than a weight-map update.

Two further points of scope. The construction is generic over the GSLT: it applies to any theory one can write down in MeTTaIL’s `language!` block [14], so one obtains a stochastic machine for the ambient calculus, for a user-defined DSL, or for the `rho` calculus, by writing the theory rather than by writing a simulator. And the weight codomain is a parameter. We treat two instances: $\mathbb{R}_{\geq 0}$, giving classical Gillespie simulation (§6); and $\mathbb{C}$, giving a quantum continuous-time Markov chain in the sense of [22] and a Gillespie-inspired trajectory simulation for it (§7). A third instance — weights in a quantale, giving a fuzzy calculus — is deliberately out of scope here; see [5].

What is new. Beyond the two distinguishing points above:

- (i) The *partition discipline* of §3, and the observation that it is not a technicality but the condition under which weighting is well defined at all — with three independent payoffs (totality of the classifier, a Boolean demand on the target logic specification, and a projective measurement in the quantum case).
- (ii) A propensity formula (§5) that separates four factors which the literature usually conflates: the refinement’s weight, a geometric factor accumulated up the context rules, redex multiplicity, and a funding gate supplied by cost accounting.
- (iii) Theorem 26: SPiM is the construction’s restriction, exactly.

- (iv) Theorem 35: the Gillespie algorithm is the quantum-jump unravelling at zero Hamiltonian and diagonal jump structure — so “Gillespie-inspired” is a theorem about degeneration, not an analogy.
- (v) The observation (§7.5) that the OSLF-generated formulae supply exactly the atomic propositions that quantum CTMC model checking otherwise has to be handed by hand.
- (vi) The recurrent-neural-network example of §8, in which the synaptic weights live in the weight map, the threshold lives in the join structure of a for-comprehension, the logistic response is emergent, and learning is a weight-map update performed by the same rewrite that performs inference.

Where this note sits. It supplies the construction that [18] cites as *Stochastic simulation for decorated ciGSLTs* and fills the placeholder section of [13]. It depends on [16] for GSLTs and OSLF, on [1] and [18] for the shape of the generated spatial-behavioural logic — which is spelled out in those papers and not yet present in the implementation branch — and on [2] for cost accounting, which enters only in §10 and only as a gate.

2 Background, compressed

We recall only what the construction uses, and forward everything else to [16].

2.1 GSLTs and their presentations

A *graph-structured lambda theory* is a triple (Σ, E, R) . The signature Σ is a lambda theory — terms may contain variables, and binding is part of the presentation, which is what distinguishes the setting from graph-enriched Lawvere theories. E is a set of equations imposing a structural congruence $\equiv$ on terms. R is a graph of rewrite rules adjoined to the structure as a Lawvere theory of graphs, with source and target maps satisfying the minimal requisite equations. The adjective *graph-structured* captures the rewrites; *lambda theory* captures term languages with variables.

Rules come in two kinds, and the distinction is load-bearing throughout this note.

Definition 1 (Base and context rules). A *base rule* is hypothesis-free:

$$r. \vdash L \rightsquigarrow R,$$

with L, R terms of some sort τ over Σ , possibly with pattern variables. A *context rule* is conditional on a rewrite:

$$c. \mid S \rightsquigarrow T \vdash C[S] \rightsquigarrow C[T],$$

for a one-hole context former C . Base rules are the ground of the dynamics; context rules are *addressing* — they say where a base rule may fire, not that anything happens.

This is MeTTaIL’s actual surface syntax for its **rewrites** block [14]. In the **rho** presentation, **Exec** and the communication rule are base; **ParCong** and **NewCong** are context rules.

An *interactive* GSLT is one whose base rules’ left-hand sides are headed by a distinguished family of formers K — the interaction cut — across which program and environment meet: application for the lambda calculus, parallel composition for CCS, π and **rho**. A *continued* interactive GSLT (ciGSLT) additionally has surfaces and continuations. Everything below is stated for ciGSLTs, though only the interactive structure is used.

2.2 Locations

Write $\partial\mathcal{T}$ for the space of one-hole contexts, in the McBride-derivative sense: a location is a pair $(\partial T, T)$ of a context and the term type. We use $\partial\mathcal{T}$ as a single object playing three roles, as in [16]: the index set of the generated modalities, the labels on the transition system, and the positions inside a term. It is this identification that lets a weight be attached to a *position* and read as a modality.

Definition 2 (Redex). Let $r. \vdash L \rightsquigarrow R$ be a base rule and P a term. A *redex* of r in P is a *selection* of a one-hole context $k \in \partial\mathcal{T}$ and a substitution σ for the pattern variables of L from the decomposition of P , such that $P \equiv k[L\sigma]$. Two redexes are identified when they select the same occurrences; interchanging two occurrences of $\equiv$ -equal subterms yields *distinct* redexes, which happen to have $\equiv$ -equal contracta. Write $\text{Red}_r(P)$ for the set of redexes of r in P , and $\text{Red}(P) = \coprod_r \text{Red}_r(P)$.

$\text{Red}_r(P)$ is finite for finite P and finitely many rules. The convention on identification is Gillespie’s, and it is the one that matters: two molecules of the same species offer two ways for a reaction to occur, not one, and a stochastic simulator that identifies them runs at half speed. In an associative–commutative setting the reactants are drawn from a multiset, so $|\text{Red}_r(P)|$ is a count of selections from a bag — $\text{In} \cdot \text{Out}$ for a binary interaction drawing one reactant from each of two species, $\binom{n}{2}$ for one drawing two from the same species. Structural congruence is used to *find* the decompositions, not to collapse them. Remark 22 records what happens when distinct redexes do share a contractum.

2.3 The generated logic

The following is generated by OSLF from the presentation and is not ours to design. We state it for the **rho** calculus — term formers $\mathbf{0}, *n, n!(Q), \text{for}(x \leftarrow n)P$, parallel composition, and $@P$, with parallel composition associative–commutative and a single communication rewrite — and note that everything transfers verbatim to any other presentation, with the connective list changing to match the formers. The presentation follows [1, §2] and [18, §3]; it is not present in the implementation branch, whose logic is still under development.

Structural connectives. One connective per term former. $\text{out}(n)\varphi$ holds of $n!(Q)$ with $Q \models \varphi$; $\text{in}(n)\varphi$ is the analogue on receipts. Because parallel composition is associative–commutative, the generated connective for it is a separating conjunction:

$$P \models \varphi \mid \psi \iff P \equiv Q \mid R \text{ with } Q \models \varphi, R \models \psi.$$

Behavioural modalities. One primitive modality $\langle K_j \rangle$ per rewrite rule *and choice of redex position*, labelled by the one-hole context in which the step fires. The index is not decoration. For **rho**, with a single interaction rule, all the information in a label is in the position: for $P \equiv K_j[\text{for}(x \leftarrow n)Q \mid n!(Q')]$ the label K_j is everything else. Thus $\exists j. \langle K_j \rangle \varphi$ and $\forall j. [K_j] \varphi$ quantify over redex positions, and a bare $\langle K \rangle$ abbreviates the existential.

Name predicates. A name is a quoted process, so

$$n \models @ \varphi \iff n = @P \text{ with } P \models \varphi,$$

which is namespace logic [15]: a namespace is described by a property rather than enumerated. This matters in §8, where synapses are picked out by structured names.

Propositional layer. Conjunction and $\top$ come from the finite-limits fragment; disjunction, implication and negation require more, and a target specification supplies them or does not [16]. §3.3 shows that weighting *forces* the choice.

Additions. A greatest fixed point $\nu X.\varphi$ with X positive; the hidden-name quantifier $\mathbf{H}x.\varphi$; graded separation $\varphi \mid_k \psi$ [1, §2.3]. We use ν only in §8.9.

Definition 3 (Depth). The depth $d(\varphi)$ of a structural formula is the maximal nesting of term-former connectives and prefixes: $d(\top) = 0$, $d(\mathbf{out}(n)\varphi) = d(\mathbf{in}(n)\varphi) = 1 + d(\varphi)$, $d(\varphi \mid \psi) = \max(d(\varphi), d(\psi))$, $d(@\varphi) = d(\varphi)$, $d(\mathbf{H}x.\varphi) = 1 + d(\varphi)$.

2.4 Ideal logic, checkable fragment

One methodological commitment governs §3. Following [1, §1.3], we distinguish the *ideal* logic, adequate for the idealised calculus and the thing that *defines* a semantic notion, from *fragments*, restricted so that model checking terminates, which *decide* it. A logic whose model checking terminates is necessarily not adequate for the whole of a Turing-complete host. The relevance here is sharp and appears at Requirement (R1): a simulator evaluates its key formulae at every step, so keys must come from a checkable fragment. What is elsewhere a methodological trade-off is here an executability constraint.

3 Refining a left-hand side

3.1 The keys

Fix a base rule r . $\vdash L \rightsquigarrow R$ with L of sort τ . The set of redexes of r is carved out by the shape of L ; we want to subdivide it. Write $L^\sharp$ for the structural formula that characterises the left-hand side pattern — for the **rho** communication rule, $L^\sharp = \mathbf{in}(n)\top \mid \mathbf{out}(n)\top$ with n existentially bound. The candidate keys are the formulae of the generated logic that *refine* $L^\sharp$.

Definition 4 (Refinement). A formula φ *refines* $L^\sharp$, written $\varphi \sqsubseteq L^\sharp$, when $\models \varphi \Rightarrow L^\sharp$. The refinements of $L^\sharp$, ordered by entailment, form the *refinement lattice* $\mathcal{R}(r)$ of the rule.

We now impose three requirements on a key set $\Phi_r \subseteq \mathcal{R}(r)$. The first is the substantial one.

Definition 5 (Partition discipline). $\Phi_r = \{\varphi_1, \dots, \varphi_m\}$ is a *partition* of r when

(P1) *Exclusivity*: $\models \neg(\varphi_i \wedge \varphi_j)$ for all $i \neq j$;

(P2) *Exhaustiveness*: $\models L^\sharp \Rightarrow \bigvee_i \varphi_i$.

Definition 6 (Requirements on a key set). A key set is *admissible* when it is a partition and, in addition:

(R1) *Decidability*: each φ_i lies in a fragment for which model checking terminates on the terms of interest.

(R2) *Locality*: $d(\varphi_i) \leq \alpha_r < \infty$ for all i , for some bound α_r fixed with the rule.

(R1) is forced by execution: a simulator must classify every enabled redex before it can sample, so a key whose evaluation may diverge is a key that stops the simulation. (R2) is forced by *efficient* execution, and pays off in Lemma 10.

3.2 What the partition buys

Proposition 7 (Classification). *Let Φ_r be a partition of r . Then the assignment*

$$\text{cls}_r : \text{Red}_r(P) \longrightarrow \Phi_r, \quad \text{cls}_r(k, \sigma) = \text{the unique } \varphi_i \text{ with } L\sigma \models \varphi_i$$

is a total function, for every term P .

Proof. $(k, \sigma) \in \text{Red}_r(P)$ gives $P \equiv k[L\sigma]$, so $L\sigma \models L^\#$ by construction of $L^\#$. Exhaustiveness gives $L\sigma \models \varphi_i$ for some i , which is totality; exclusivity gives at most one such i , which is single-valuedness. Well-definedness on $\equiv$ -classes holds because satisfaction is $\equiv$ -invariant, the generated connectives being generated from Σ modulo E . $\square$

This is exactly what fails without the discipline, and it fails in two directions. Without exhaustiveness the weighting is partial: some enabled redexes have no rate, and one must invent a convention (fire them at rate zero? at a default rate? refuse to run?) that is not part of the specification. Without exclusivity the weighting is multi-valued, and one must invent an *aggregation* (take the most specific key, if a unique minimum exists; sum the weights of all satisfied keys; take the maximum). Each convention is defensible and each yields a different simulator from the same specification, which is precisely the situation a specification exists to prevent.

Remark 8 (Completing a partition is cheap). The discipline is less onerous than it looks. Given any pairwise-exclusive family $\{\varphi_1, \dots, \varphi_m\}$, the family $\{\varphi_1, \dots, \varphi_m, \varphi_\perp\}$ with $\varphi_\perp := L^\# \wedge \neg \bigvee_i \varphi_i$ is a partition, and a modeller who wants the unlisted redexes inert simply sets $w(\varphi_\perp) = 0$. Exclusivity is the real constraint; exhaustiveness is a completion. In practice, keys of the form “the redex is a communication on channel n ,” indexed by distinct n , are exclusive for free.

3.3 The partition forces a Boolean target specification

Proposition 9 (Boolean demand). *Stating the partition discipline requires negation, conjunction and disjunction in the generated logic. Consequently a weighted GSLT cannot be specified over a target logic specification carrying only the finite-limits fragment.*

Proof. Immediate from (P1) and (P2): (P1) mentions $\neg$ and $\wedge$, (P2) mentions $\Rightarrow$ and $\bigvee$. Finite limits supply $\wedge$ and $\top$ only [16]. $\square$

This is worth pausing on, because it is a convergence rather than an inconvenience. The omnibus identifies possession of something like a negation and a conjunction as exactly the condition under which a generated logic enjoys an adequacy theorem, and notes that Hypercube-style members of the OSLF family, which generate type systems, offer no such thing [16]. Proposition 9 says that the weighting apparatus makes the same demand for an independent reason. The moral: *one cannot weight a theory whose generated logic is too weak to be adequate for it*. A weighting is a claim about which behaviours are distinguishable enough to be priced differently, and that claim needs the same expressive power as the claim that they are distinguishable at all.

Requirement (R1) then pulls the other way, since a terminating fragment is necessarily inadequate for a Turing-complete host. The two are not in conflict, because they apply to different artifacts: adequacy is a property of the ideal logic in which the specification is *stated*, and termination is a property of the fragment in which the keys are *evaluated*. A modeller writes a partition in the ideal logic, then checks that the particular keys chosen happen to lie in a checkable fragment. §8 exhibits keys of depth 2, for which the check is trivial.

3.4 Locality

Lemma 10 (Locality of reclassification). *Let all keys of all rules have depth at most α , let $P \equiv k[L\sigma]$ and let $P' \equiv k[R\sigma]$ be the result of firing that redex. Then for any redex (k', σ') of any rule surviving in P' whose position is at distance greater than $\alpha + |L| + |R|$ from the fired position in the term tree, $\text{cls}(k', \sigma')$ is unchanged.*

Proof. A formula of depth α evaluated at a position inspects only the subterm within α nested formers of that position, and the separating conjunction inspects a decomposition of that subterm, not of the whole. The rewrite replaces $L\sigma$ by $R\sigma$ and leaves k pointwise fixed, so any window of radius α disjoint from the replaced material sees an identical subterm. The offset $|L| + |R|$ accounts for windows that overlap the replaced region. $\square$

The point is algorithmic. Recomputing cls over the whole term at every step makes the simulator quadratic in term size; Lemma 10 licenses an incremental propensity update in the style of Gibson and Bruck’s next-reaction method [6], touching only a bounded neighbourhood of the fired redex. This is where (R2) earns its place, and it is also the reason to prefer shallow keys where the modelling permits.

4 Weighted rewrite systems

4.1 The weight codomain

Weights are drawn from a commutative semiring $\mathcal{K}$. We treat two instances:

$$\mathcal{K} = \mathbb{R}_{\geq 0} = (\mathbb{R}_{\geq 0}, +, \cdot, 0, 1) \quad \text{and} \quad \mathcal{K} = \mathbb{C}.$$

Remark 11 (Rates, not probabilities). We take the real codomain to be $\mathbb{R}_{\geq 0}$ rather than the unit interval. A Gillespie waiting time is $\tau = a_0^{-1} \ln(1/u)$ and therefore carries units of time; a weight is a rate, with units of inverse time, and there is nothing to normalise it against. The unit interval is available as a derived presentation: fixing a global clock rate $\lambda \geq a_0$ and setting $p_i = a_i/\lambda$ gives the uniformised discrete-time chain, whose jump probabilities do live in $[0, 1]$ and whose sojourn is a fixed exponential of rate λ . Uniformisation is a good implementation strategy and a bad specification, because λ must dominate the propensity of every reachable configuration and the dynamic weight maps of §4.4 make that bound a global proof obligation rather than a local one. We specify in $\mathbb{R}_{\geq 0}$ and let an implementation uniformise if it wishes.

For $\mathbb{C}$ we do require $|z| \leq 1$ on each entry, for the reason given at Definition 30: an entry becomes a jump operator, and the resulting semigroup must be trace non-increasing.

4.2 Weight maps and augmented rules

Definition 12 (Weight map). Let (Σ, E, R) be a GSLT with admissible key sets Φ_r for each base rule r . A *weight map* is a function

$$w : \coprod_r \Phi_r \longrightarrow \mathcal{K}.$$

Write $\mathcal{W}$ for the set of weight maps. Since each Φ_r is finite and there are finitely many rules, $\mathcal{W} \cong \mathcal{K}^N$ for a fixed N .

Definition 13 (Configuration). A *configuration* is a pair $\mathbf{c} = (P, w)$ with P a term modulo $\equiv$ and $w \in \mathcal{W}$. Write $\mathcal{C}$ for the set of configurations.

The phrase “a map instance is part of a state specification” is Definition 13, and everything downstream turns on it.

Definition 14 (Augmented rules). An *augmented base rule* is

$$r. \vdash L \rightsquigarrow R [w_r] \{ \varphi_1 \Rightarrow (v_1, u_1), \dots, \varphi_m \Rightarrow (v_m, u_m) \}$$

where $\{\varphi_i\}$ is an admissible key set for r , $v_i \in \mathcal{K}$ are the initial weights, and $u_i : \mathcal{W} \rightarrow \mathcal{W}$ are *update functions* on the whole map. An *augmented context rule* is

$$c. \mid S \rightsquigarrow T \vdash C[S] \rightsquigarrow C[T] [g_c] \{ f_c \}$$

with $g_c \in \mathbb{R}_{\geq 0}$ a *geometric factor* and $f_c : \mathcal{W}^* \times \mathcal{W} \rightarrow \mathcal{W}$ a *fold function*.

The initial weight map is $w_0(\varphi_i) = v_i$; the leading $[w_r]$ is a per-rule scalar, convenient in practice and formally absorbable into the v_i , which is how we treat it below.

4.3 Why context rules carry a factor and not a propensity

The design decision that most distinguishes Definition 14 from the obvious alternative is that context rules do not have propensities of their own.

The obvious alternative gives every rule, base and context alike, a weight and lets the simulator select among all of them. It double-counts. A step of the transition system is a base rule firing *somewhere*; the context rules composing the path from the root to that somewhere are the address of the firing, not additional events. In the vocabulary of the energetics reading of OSLF-decorated GSLTs, structural rules are addressing, not dynamics, and incur no primitive cost. Selecting a context rule and then a base rule as though they were independent draws makes a single event into two.

So context rules contribute multiplicatively, to the propensity of the base firing they address:

Definition 15 (Geometric factor). Let $k \in \partial\mathcal{T}$ be a position, obtained by composing context rules $c_1, \dots, c_p$ from the root inwards. Its *geometric factor* is $g(k) = \prod_{q=1}^p g_{c_q} \in \mathbb{R}_{\geq 0}$.

Setting every $g_c = 1$ recovers “addressing is free” exactly, and is the default we recommend. The reason to permit $g_c \neq 1$ is that it is the natural home for a *spatial* bias: g is a weighting on the reduction graph’s edges that depends only on where in the term the interaction surfaces are, as against $w(\varphi)$, which depends on what is being transferred. This is the geometric-versus-matter split of the gravitation correspondence [3] appearing here as an implementation detail, and we flag it as such without pursuing it.

4.4 Dynamics

Definition 16 (Weighted transition). Let $\mathbf{c} = (P, w)$ and let $(k, \sigma) \in \text{Red}_r(P)$ with $\text{cls}_r(k, \sigma) = \varphi_i$, the position k composed of context rules $c_1, \dots, c_p$. Then $\mathbf{c}$ steps to $\mathbf{c}' = (P', w')$ where

$$P' \equiv k[R\sigma], \quad w' = f_{c_1}(\vec{\omega}_1, f_{c_2}(\vec{\omega}_2, \dots f_{c_p}(\vec{\omega}_p, u_i(w)) \dots)),$$

the fold functions being applied from the redex outwards, with $\vec{\omega}_q$ the maps arising from the sibling subterms at level q .

In the common case where every f_c is the projection onto its second argument — “the context does not modify the map” — this collapses to $w' = u_i(w)$, and we write it so below.

Definition 17 (The coalgebra). Let $\mathcal{K}^{(-)}$ denote the free- $\mathcal{K}$ -semimodule monad, $\mathcal{K}^{(X)}$ being finitely supported functions $X \rightarrow \mathcal{K}$. A weighted GSMT determines a coalgebra

$$\gamma : \mathcal{C} \longrightarrow \mathcal{K}^{(\mathcal{C})}, \quad \gamma(\mathbf{c})(\mathbf{c}') = \sum_{\substack{(r,k,\sigma) \\ \text{stepping } \mathbf{c} \rightarrow \mathbf{c}'}} w(\text{cls}_r(k, \sigma)) \cdot g(k).$$

This is the target of the whole construction: a decorated ciGSMT plus an adequate spatial-behavioural logic plus a weight-map decoration gives a $\mathcal{K}$ -weighted coalgebra. Assembling the decorations of a fixed theory into a category and taking the Grothendieck construction gives the fibred form; the simulator is a functor out of it. We do not need the fibred statement below and record it only to fix the picture.

Remark 18 (Sum over derivations, not over targets). The sum in Definition 17 runs over *derivations* that reach $\mathbf{c}'$, not merely over the target. In $\mathbb{R}_{\geq 0}$ this is the familiar statement that parallel routes to the same state add their rates. In $\mathbb{C}$ it is the interference condition, and it is the reason the quantum case is not a routine relabelling; see §7.3.

5 Propensity

The propensity of a rule at a refinement is where the construction is most easily got wrong, because four independent factors are involved and the literature usually presents some of them fused. We separate them.

Definition 19 (Propensity). Let $\mathbf{c} = (P, w)$ be a configuration, r a base rule and $\varphi \in \Phi_r$. Put

$$K(r, \varphi, P) = \{ (k, \sigma) \in \text{Red}_r(P) : \text{cls}_r(k, \sigma) = \varphi \}.$$

The *propensity* of (r, φ) at $\mathbf{c}$ is

$$\mathbf{a}(r, \varphi, \mathbf{c}) = w(\varphi) \cdot \sum_{(k, \sigma) \in K(r, \varphi, P)} g(k) \cdot \chi(r, k, \sigma),$$

where $\chi \in \{0, 1\}$ is the funding gate of §10, identically 1 in the absence of cost accounting. The *total propensity* is $\mathbf{a}_0(\mathbf{c}) = \sum_r \sum_{\varphi \in \Phi_r} \mathbf{a}(r, \varphi, \mathbf{c})$.

The four factors:

$w(\varphi)$ — **the rate constant**.

What kind of interaction this is, priced. Dynamic: this is the component update functions rewrite.

$g(k)$ — **the geometric factor**.

Where the interaction is, priced (Definition 15). Default 1.

The cardinality of the sum — multiplicity.

How many ways this interaction is available. This is Gillespie's h_j , the combinatorial count of distinct reactant tuples [7], and it is the factor most often omitted in implementations, with the consequence that a term with fifty available redexes of a kind fires them no faster than a term with one.

χ — **the funding gate**.

Whether the interaction can be afforded (§10).

Proposition 20 (Multiplicity is well defined and computable). *For finite P , $|K(r, \varphi, P)|$ is finite, invariant under $\equiv$, and computable by enumerating matches of L in P modulo $\equiv$ and evaluating cls_r at each.*

Proof. Finiteness and $\equiv$ -invariance are Definition 2 together with Proposition 7; computability of the classification is (R1). In the ρ presentation the multiset structure of parallel composition is carried explicitly — MeTTaIL represents PPar as a hash bag [14] — so “matches modulo associativity and commutativity” is a bag operation rather than a search over permutations. $\square$

Proposition 21 (Total propensity is a sum over funded redexes). *With admissible key sets,*

$$\mathbf{a}_0(P, w) = \sum_r \sum_{(k, \sigma) \in \text{Red}_r(P)} w(\text{cls}_r(k, \sigma)) \cdot g(k) \cdot \chi(r, k, \sigma).$$

That is: every enabled redex contributes exactly once, at the rate of its own class.

Proof. The sets $K(r, \varphi, P)$ for $\varphi \in \Phi_r$ partition $\text{Red}_r(P)$, by Proposition 7. Exchange the order of summation. $\square$

Proposition 21 is the payoff of the partition discipline in its most practical form. It says that the propensity computation may be organised *by redex* rather than *by key* — walk the term, find each redex, classify it, add its rate — with no risk of a redex being counted twice or missed. Without exclusivity the first walk over-counts; without exhaustiveness it under-counts. Neither error is visible in a trace; both silently distort the sampled distribution.

Remark 22 (Symmetry, and distinct redexes with a common contractum). Definition 2 counts selections from a multiset, which gives the symmetry factors of chemical kinetics for free: the $\binom{n}{2}$ of a dimerisation of like molecules is the number of unordered pairs drawn from a bag of n , and the In · Out of a binary interaction is the number of ways of drawing one reactant from each side. It does *not* handle the correction SPiM applies for an input and an output competing within a single summation, because the ρ calculus has no summation operator; see the proof of Theorem 26.

Distinct redexes may nonetheless have $\equiv$ -equal contracta — m indistinguishable pending messages on a channel give m redexes and one target. Classically this is invisible: the m contributions add, so the rate to the target is m times the base rate, which is what Proposition 21 says. In the complex setting it is *not* invisible, because m amplitudes add before being squared; see Proposition 33 and Remark 52.

6 Real weights: the Gillespie construction

6.1 The simulator

Definition 23 (Weighted GSLT simulator, $\mathcal{K} = \mathbb{R}_{\geq 0}$). Given an initial configuration $\mathbf{c}_0 = (P_0, w_0)$ and $t := 0$, repeat:

1. Compute $\mathbf{a}(r, \varphi, \mathbf{c})$ for all (r, φ) and $\mathbf{a}_0(\mathbf{c})$. If $\mathbf{a}_0 = 0$, halt.
2. Draw $u_1, u_2 \sim U(0, 1)$. Set $\tau := \mathbf{a}_0^{-1} \ln(1/u_1)$.
3. Select (r, φ) with probability $\mathbf{a}(r, \varphi, \mathbf{c})/\mathbf{a}_0(\mathbf{c})$, then select a position $(k, \sigma) \in K(r, \varphi, P)$ with probability $g(k)\chi(k)/\sum_{k'} g(k')\chi(k')$, using u_2 .
4. Fire: $P := k[R\sigma]$, and $w :=$ the fold of Definition 16, which in the default case is $u_i(w)$.

5. $t := t + \tau$.

Steps 1–3 and 5 are Gillespie’s direct method [7] verbatim. Step 4 is the addition, and the only one.

Theorem 24 (Markovianity on configurations). *The process of Definition 23 is a time-homogeneous continuous-time Markov chain on $\mathcal{C}$, with generator*

$$Q(\mathbf{c}, \mathbf{c}') = \gamma(\mathbf{c})(\mathbf{c}') \quad (\mathbf{c}' \neq \mathbf{c}), \quad Q(\mathbf{c}, \mathbf{c}) = -\mathbf{a}_0(\mathbf{c}),$$

for γ the coalgebra of Definition 17. It is not in general a Markov chain on terms alone.

Proof. The propensity of Definition 19 is a function of $\mathbf{c} = (P, w)$ and of nothing else — in particular not of t or of the history — and the successor configuration is determined by $\mathbf{c}$ together with the selected redex. Waiting times are exponential with parameter $\mathbf{a}_0(\mathbf{c})$, which depends only on $\mathbf{c}$. This is the definition of a time-homogeneous CTMC with the stated generator. For the negative claim, it suffices to exhibit a theory and two runs reaching the same term with different maps, which we do at Example 25. $\square$

Theorem 24 is the justification for Definition 13, and it is worth being explicit about the shape of the argument, because the design could have gone otherwise. If weight maps could change but were *not* part of the state, the process on terms would be non-Markovian: the rate of a transition out of P would depend on how P was reached. One would then be in the world of semi-Markov or generalised semi-Markov processes, with no exact simulation algorithm and no CTMC to model check. Putting the map in the state restores exactness at the price of a larger state space. That price is real — $\mathcal{C}$ is $\mathcal{T} \times \mathcal{K}^N$, a continuum — and §6.3 says when it is affordable.

Example 25 (Dynamic weights are not a term-level phenomenon). Take the **rho** theory with a single communication rule and keys $\Phi = \{\varphi_a, \varphi_b, \varphi_\perp\}$, where φ_x says the redex is a communication on the channel x . Let the update functions act on $w(\varphi_a)$ alone, by

$$u_a : w(\varphi_a) \mapsto w(\varphi_a) + 1, \quad u_b : w(\varphi_a) \mapsto 2w(\varphi_a),$$

which do not commute. Put $Q = \text{for}(z \leftarrow a)\{\mathbf{0}\} \mid a!(\mathbf{0})$ and

$$P_0 = \text{for}(y \leftarrow a)\{Q\} \mid a!(\mathbf{0}) \mid \text{for}(y \leftarrow b)\{Q\} \mid b!(\mathbf{0}).$$

P_0 has exactly two redexes and both firing orders consume both, reaching the same term Q . But starting from $w_0(\varphi_a) = 1$, firing a first reaches Q with $w(\varphi_a) = (1+1) \cdot 2 = 4$, and firing b first reaches it with $w(\varphi_a) = 1 \cdot 2 + 1 = 3$. Q carries a single a -redex, so the two runs sit at the same term with expected waiting times $1/4$ and $1/3$. No rate function of the term alone reproduces both.

6.2 SPiM is the static, name-keyed, ungeometric restriction

Theorem 26 (SPiM conservativity). *Instantiate the construction with*

- (a) *the GSLT of the π -calculus, presented in MeTTaIL;*
- (b) *key set $\Phi_{\text{COMM}} = \{\varphi_n : n \in \mathcal{N}\} \cup \{\varphi_\perp\}$, where φ_n holds of a redex just when it is a communication on the channel n ;*
- (c) *$w(\varphi_n) = \rho(n)$ the SPiM channel rate, $w(\varphi_\perp) = 0$;*

(d) all update functions the identity, all $g_c = 1$, all fold functions the second projection, and $\chi \equiv 1$.

Then Φ_{COMM} is admissible, the map is constant along every run, and for every process P the apparent rate of channel n computed by Definition 19 coincides with SPiM's:

$$\mathbf{a}(\text{COMM}, \varphi_n, P) = \rho(n) \cdot \text{In}_n(P) \cdot \text{Out}_n(P),$$

with $\text{In}_n, \text{Out}_n$ the counts of unguarded inputs and outputs on n . Hence the two simulators induce the same CTMC and the same trace distribution.

Proof. Admissibility. Distinct channels give disjoint sets of communication redexes, so (P1) holds; $\varphi_\perp$ completes (P2) by Remark 8. Each φ_n is expressible as $\text{in}(n)\top \mid \text{out}(n)\top$ at depth 2, so (R1) and (R2) hold with $\alpha = 2$.

Staticity. All $u_i = \text{id}$ and all folds are second projections, so Definition 16 gives $w' = w$.

The rate. With $g \equiv 1$ and $\chi \equiv 1$, Definition 19 reduces to $\rho(n) \cdot |K(\text{COMM}, \varphi_n, P)|$. A communication redex on n is a choice of one unguarded input on n and one unguarded output on n in the parallel decomposition of P , taken modulo $\equiv$; the count is therefore $\text{In}_n(P) \cdot \text{Out}_n(P)$. This is SPiM's activity for n [20], whose general form subtracts $\sum_s \text{In}_n^s(P) \cdot \text{Out}_n^s(P)$ over summations s to exclude an input and an output competing within the same choice; the presentation at (a) is the summation-free fragment, in which the correction term is empty.

Coincidence of the chains. Both simulators are Gillespie direct-method samplers over the same propensity function on the same state space (terms, the map being constant), so their generators agree and their trace distributions agree. $\square$

Corollary 27 (Strictness). *The construction is strictly more expressive than SPiM in two independent directions. Relaxing (b) admits keys not of the form φ_n — for instance $\text{in}(n)\top \mid \text{out}(n)\text{@}\varphi$, “a communication on n whose payload names a process satisfying φ ,” which prices a transition by what is transferred and has no SPiM counterpart. Relaxing (d) admits $u_i \neq \text{id}$, and Example 25 exhibits a term-level behaviour no static-rate calculus reproduces.*

We stress that the first direction is not merely a convenience. In a reflective calculus a name is a quoted process, so a key may inspect the name — $n \models \text{@}\varphi$ — and hence describe a *namespace* rather than a channel [15]. “Communications on any channel in the namespace of synapses of neuron i ” is a single key of bounded depth, where the name-keyed presentation would require one key per channel and would have to be regenerated when the network grew. §8.9 uses exactly this.

6.3 Finiteness

Nothing above requires $\mathcal{C}$ to be finite; simulation only requires each step to be computable, and each step is. Model checking is another matter, and so is the quantum construction of §7, which builds an operator on a space spanned by reachable terms. We therefore record the condition:

Definition 28 (Bounded configuration space). A weighted GSLT with initial configuration $\mathbf{c}_0$ is *term-finite* when the set of terms reachable from P_0 is finite modulo $\equiv$, and *finite* when in addition the set of weight maps reachable from w_0 is finite.

Term-finiteness fails for a Turing-complete host in general and holds for the class of interest in §8, where it is a consequence of boundedness of a marked graph (Lemma 46). Map-finiteness fails as soon as an update function is a nontrivial contraction or dilation, as Hebbian potentiation is; the standard remedies are to quantise the weights, to saturate them at a ceiling, or to model check the term-marginal only. We adopt saturation in §8.7, which is in any case what a bounded synapse does.

7 Complex weights: a quantum continuous-time Markov chain

We now take $\mathcal{K} = \mathbb{C}$ with $|z| \leq 1$ on entries. Everything in §§3–5 is unchanged: the keys are the same formulae, the partition discipline is the same, the classification is the same. What changes is what one does with the numbers.

7.1 The space

Definition 29 (Configuration space). Let $\mathbf{c}_0 = (P_0, w_0)$ be term-finite (Definition 28) and let $\mathcal{B}$ be the set of terms reachable from P_0 modulo $\equiv$. The *configuration Hilbert space* is $\mathcal{H} = \ell^2(\mathcal{B})$, with orthonormal basis $\{|P\rangle\}_{P \in \mathcal{B}}$. A state is a density operator ρ on $\mathcal{H}$.

Two comments. The quotient by $\equiv$ is essential and not bookkeeping: $\mathcal{B}$ must be a set of *states*, so $P \mid Q$ and $Q \mid P$ must be the same basis vector, or the parallel composition of two independent components would double the dimension and destroy normalisation. Term-finiteness is required for $\mathcal{H}$ to be finite-dimensional, which the model checking of §7.5 assumes; §8 supplies a class where it holds.

The weight map is not part of $\mathcal{H}$. It parameterises the *generator*, and the update functions switch the generator at a jump. This is the honest reading of dynamic maps in the quantum case: along a trajectory the evolution is piecewise-Lindbladian, with the pieces separated by jumps.

7.2 Jumps from refinements, coherence from equations

Definition 30 (Jump operators). For a base rule r and $\varphi \in \Phi_r$ with $w(\varphi) = z$, define

$$L_{r,\varphi} = z \sum_{P \in \mathcal{B}} \sum_{(k,\sigma) \in K(r,\varphi,P)} \sqrt{g(k) \chi(r,k,\sigma)} \mid k[R\sigma] \rangle \langle P \mid.$$

$L_{r,\varphi}$ is bounded on a finite-dimensional $\mathcal{H}$, and the geometric factor enters under a square root because it will be squared back when the operator is paired with its adjoint. The requirement $|z| \leq 1$ together with a normalisation of g makes $\sum_{r,\varphi} L_{r,\varphi}^\dagger L_{r,\varphi}$ bounded, which is what the semigroup needs.

Where does the coherent part come from? The rewrites are directed and irreversible, so they are naturally dissipative; the *equations* are symmetric and cost-free, so they are naturally unitary. We record this as a design principle rather than a theorem, because it depends on a choice in the presentation:

Definition 31 (Hamiltonian from the equational theory). Let $E_h \subseteq E$ be a distinguished subset of the equations of the GSLT, presented with amplitudes $h_e \in \mathbb{R}$, each $e \in E_h$ relating $P \equiv_e P'$ with $P \not\equiv P'$ in the chosen normal form. Put

$$H = \sum_{e \in E_h} h_e \sum_{P \equiv_e P'} (|P'\rangle \langle P| + |P\rangle \langle P'|),$$

a Hermitian operator on $\mathcal{H}$.

The point is that a GSLT already comes with two kinds of relation on terms — symmetric, cost-free equations and directed, costed rewrites — and a quantum reading has exactly two slots to fill. Equations quotiented into the basis (as $\equiv$ is, above) contribute nothing; equations left un-quotiented and given amplitudes are coherent hopping. Most presentations quotient everything, so $H = 0$, which is the case Theorem 35 concerns.

Definition 32 (The QCTMC of a weighted GSLT). The generator is the Lindbladian

$$\mathcal{L}(\rho) = -i[H, \rho] + \sum_{r, \varphi} \left(L_{r, \varphi} \rho L_{r, \varphi}^\dagger - \frac{1}{2} \{ L_{r, \varphi}^\dagger L_{r, \varphi}, \rho \} \right),$$

and the evolution is $\rho(t) = e^{t\mathcal{L}}\rho(0)$. The pair $(\mathcal{H}, \mathcal{L})$ is a quantum continuous-time Markov chain in the sense of [22].

7.3 Interference

Proposition 33 (When amplitudes interfere). $L_{r, \varphi}$ maps $|P\rangle$ to a superposition whose coefficient at $|P''\rangle$ is $z \sum \sqrt{g(k)\chi(k)}$ summed over those redexes $(k, \sigma) \in K(r, \varphi, P)$ with $k[R\sigma] \equiv P''$. Interference between two distinct redexes of the same class therefore occurs exactly when they have $\equiv$ -equal contracta — that is, exactly at a closed local diamond of the rewrite relation.

Proof. Immediate from Definition 30 and orthonormality of the basis. $\square$

This is the substantive difference between the two codomains. In $\mathbb{R}_{\geq 0}$, parallel routes to the same state add their rates and nothing surprising happens. In $\mathbb{C}$ they add amplitudes, so two routes carrying z and $-z$ cancel and the target is unreachable despite both routes being individually enabled. Local confluence of the rewrite system is thus not an incidental structural property in the quantum setting but the precise condition under which interference is possible at all: a rewrite relation with no closed diamonds gives a jump operator with at most one contribution per matrix entry, hence a chain in which no cancellation can occur, hence one whose $|z|^2$ marginal is a classical CTMC. *Concurrency is what makes the quantum reading non-trivial.*

7.4 Gillespie as a degenerate quantum jump

The simulation of $\rho(t) = e^{t\mathcal{L}}\rho(0)$ by trajectories is the quantum-jump or Monte-Carlo-wavefunction method [4, 12]. Its structure is the reason “Gillespie-inspired” is more than an analogy.

Definition 34 (Trajectory simulator, $\mathcal{K} = \mathbb{C}$). Put $H_{\text{eff}} = H - \frac{i}{2} \sum_{r, \varphi} L_{r, \varphi}^\dagger L_{r, \varphi}$. From $|\psi(0)\rangle = |P_0\rangle$ and $t := 0$, repeat:

1. Draw $u \sim U(0, 1)$.
2. Evolve $|\tilde{\psi}(s)\rangle = e^{-iH_{\text{eff}}s}|\psi\rangle$, whose norm decreases, until the first τ with $\|\tilde{\psi}(\tau)\|^2 = u$.
3. Select a jump (r, φ) with probability proportional to $\|L_{r, \varphi}\tilde{\psi}(\tau)\|^2$, and normalise:

$$|\psi\rangle := L_{r, \varphi}\tilde{\psi}(\tau) / \|L_{r, \varphi}\tilde{\psi}(\tau)\|.$$

4. Apply the update function u_φ to w , rebuilding L and H_{eff} ; set $t := t + \tau$.

Averaging $|\psi\rangle\langle\psi|$ over trajectories reproduces $\rho(t)$.

Theorem 35 (The SSA is the quantum-jump algorithm at $H = 0$). *Suppose $H = 0$, every weight $w(\varphi)$ is real and non-negative, and the jump structure is diagonal: for each P , each (r, φ) and each pair of distinct redexes in $K(r, \varphi, P)$, the contracta are distinct. Let $|\psi\rangle = |P\rangle$ be a basis state. Then Definition 34 reduces to Definition 23. Precisely:*

$$(i) \quad \|\tilde{\psi}(s)\|^2 = e^{-\Gamma(P)s} \text{ with } \Gamma(P) = \sum_{r, \varphi} \|L_{r, \varphi}|P\rangle\|^2 = \mathbf{a}_0(P, w);$$

(ii) hence τ is exponential with parameter $\mathbf{a}_0$, as in step 2 of Definition 23;

(iii) the jump probabilities of step 3 are $\mathbf{a}(r, \varphi)/\mathbf{a}_0$, and within a jump the position is selected with probability $g(k)\chi(k)/\sum_{k'} g(k')\chi(k')$;

(iv) the post-jump state is again a basis state, so the reduction propagates.

Proof. Under diagonality, $L_{r,\varphi}|P\rangle = w(\varphi) \sum_{(k,\sigma)} \sqrt{g\chi} |k[R\sigma]\rangle$ has pairwise-orthogonal terms, so $\|L_{r,\varphi}|P\rangle\|^2 = w(\varphi)^2 \sum_{(k,\sigma)} g(k)\chi(k)$. Absorbing the square (equivalently, taking $w(\varphi)$ in Definition 30 to be the amplitude whose square is the rate) gives $\mathbf{a}(r, \varphi, P)$ of Definition 19, and summing over (r, φ) gives (i) with $\Gamma = \mathbf{a}_0$. With $H = 0$, $H_{\text{eff}} = -\frac{i}{2} \sum L^\dagger L$, and $\sum L^\dagger L$ acts on $|P\rangle$ as the scalar $\Gamma(P)$ by the same orthogonality, so $e^{-iH_{\text{eff}}s}|P\rangle = e^{-\Gamma(P)s/2}|P\rangle$ and $\|\tilde{\psi}(s)\|^2 = e^{-\Gamma(P)s}$. Solving $e^{-\Gamma\tau} = u$ gives $\tau = \Gamma^{-1} \ln(1/u)$, which is (ii). Statement (iii) is the ratio of the norms just computed, and the within-jump selection follows from the orthogonality of the contracta, which makes $L_{r,\varphi}|P\rangle$ a state whose measurement in the basis yields $k[R\sigma]$ with probability $g(k)\chi(k)/\sum g\chi$. Since a measurement of a superposition of basis states in that basis returns a basis state, (iv) follows and the argument applies again at the next step. $\square$

Two things are worth extracting. First, the exact respect in which the quantum simulator is Gillespie-*inspired* rather than Gillespie: the waiting time is drawn from the decaying norm $\|\tilde{\psi}(\tau)\|^2$, which is an exponential only in the degenerate case; in general the norm decays non-exponentially because H_{eff} moves amplitude between basis states before any jump occurs, so the sojourn distribution is state-dependent in a way no classical propensity reproduces. Sampling the decaying norm *is* the Gillespie step, generalised. Second, the hypotheses of the theorem name exactly the two features that make a system quantum: a nonzero Hamiltonian (coherent evolution between jumps) and a non-diagonal jump structure (interference, Proposition 33). Drop both and one is running SSA.

7.5 The generated logic supplies the atomic propositions

Model checking a QCTMC requires atomic propositions, and in the literature these are supplied externally: [22] takes them to be constraints $\text{Tr}(\Pi\rho(t)) \bowtie p$ for a projector Π handed to the checker, verifies them by real root isolation of exponential polynomials, and builds signal temporal logic on top; the successor [23] does the same for continuous stochastic logic. Choosing the projectors is the modeller's problem and it is not a small one, since a projector is a subspace of a space whose basis is a set of program states.

Here they are already present, and the partition discipline is what makes them well behaved.

Proposition 36 (Refinement classes are a projective measurement). *For each formula φ of the generated logic put $\Pi_\varphi = \sum_{P \in \mathcal{B}, P \models \varphi} |P\rangle\langle P|$. Then each Π_φ is an orthogonal projector, and for an admissible key set Φ_r the family $\{\Pi_\varphi\}_{\varphi \in \Phi_r}$ is a family of mutually orthogonal projectors summing to $\Pi_{L^\#}$; extending Φ_r by $\neg L^\#$ gives a resolution of the identity, hence a projective measurement on $\mathcal{H}$.*

Proof. Π_φ is a sum of rank-one projectors onto distinct basis vectors, hence an orthogonal projector. Exclusivity (P1) gives $\Pi_{\varphi_i} \Pi_{\varphi_j} = 0$ for $i \neq j$, since no basis vector lies in both sums; exhaustiveness (P2) gives $\sum_i \Pi_{\varphi_i} = \Pi_{L^\#}$, since every basis vector satisfying $L^\#$ appears in exactly one sum. Adding $\Pi_{\neg L^\#} = I - \Pi_{L^\#}$ completes the resolution. $\square$

This is the third payoff of the partition discipline, and the least expected one. A partition of a rule's left-hand side, imposed in §3 for the entirely classical reason that otherwise the

classification is not a function, turns out under the complex reading to be a quantum measurement — the outcomes being “which kind of interaction is available here.” The requirement that keys be Boolean-complete (Proposition 9) is the requirement that the measurement be complete. Given Proposition 36, the atomic propositions of [22, 23] are available for every formula of the generated logic, at no modelling cost: one writes $\text{Tr}(\Pi_\varphi \rho(t)) \bowtie p$ and the checker’s machinery applies unchanged.

Remark 37 (What is not claimed). We do not claim that the spatial–behavioural logic and the temporal logic of [22] combine into a single adequate logic for weighted GSLTs. The generated logic speaks about terms; STL and CSL speak about the time evolution of $\text{Tr}(\Pi \rho(t))$; here the former supplies the Π to the latter and nothing more. A genuinely combined logic — context-labelled modalities interleaved with time-bounded until — is the obvious next question and is left open in §11.

8 Worked example: a recurrent neural network

8.1 The design decision that makes the example an example

One can always model a neural network in a process calculus by writing the arithmetic into the payloads: send a real number down a channel, multiply it by a stored weight, sum, squash, forward. Nothing in this note would be doing any work; the calculus would be a programming language and the network a program written in it.

We do the opposite. The synaptic efficacies are *not* in the terms. They are the values of the weight map, keyed by spatial–behavioural formulae picking out the synapses. The consequences are that firing is a stochastic event rather than an arithmetic one, that the network’s response nonlinearity is a property of the chain rather than a function written down anywhere, and — the reason for choosing this example — that synaptic plasticity is literally the update functions of Definition 14. A network that learns and a network that infers are the same weighted GSLT running the same transition relation.

8.2 Neurons are persistent for-comprehensions

Definition 38 (Neuron). Let neuron i have presynaptic channels $a_{1i}, \dots, a_{k_i i}$ (its dendrites), an axonal channel b_i , and postsynaptic targets $\text{post}(i)$. Its *threshold presentation at θ_i* is the persistent for-comprehension

$$N_i \triangleq \text{for} \left(\underbrace{x_{s_1} \Leftarrow a_{s_1 i} \ \& \ \dots \ \& \ x_{s_\theta} \Leftarrow a_{s_\theta i}}_{\text{one group per } \theta\text{-subset } S=\{s_1, \dots, s_\theta\}} ; \dots \right) \{ \prod_{j \in \text{post}(i)} a_{ij}!(\text{spk}) \},$$

with one join group per θ_i -element subset of the dendrites, & joining within a group and ; separating groups. The network is $\text{Net} \triangleq \text{new } \vec{a} \text{ in } \{ \prod_i N_i \mid M_0 \}$, where M_0 is an initial multiset of pending spikes.

Three points about this definition, in increasing order of importance.

The receipts are *persistent* ($\Leftarrow$, not $\leftarrow$). A linear receipt evaporates after one firing, so a network built from linear receipts models a single sweep, not a network. The same distinction was load-bearing in the knot encoding of [10], and for the same reason: persistence is what makes the object a standing structure rather than a trace through one. Rholang receipts are linear-or-persistent as a whole, so the pending spikes are still consumed per firing while the neuron itself survives.

Recurrence is not an additional construct. It is the statement that b_i — equivalently, some a_{ij} for $j \in \text{post}(i)$ — feeds a dendrite of a neuron from which i is reachable, i itself

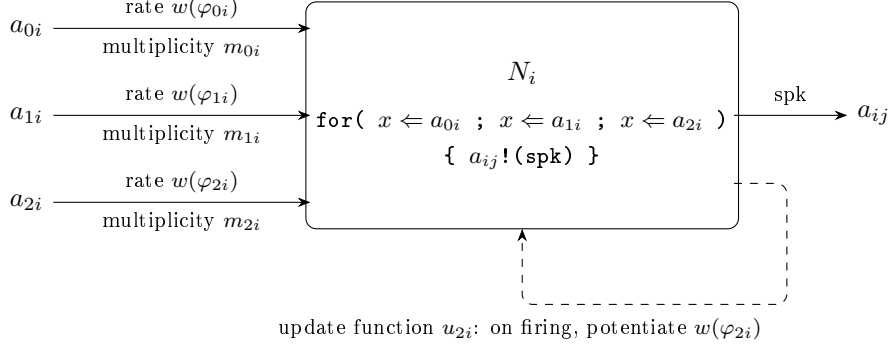


Figure 1: A neuron. The three factors of Definition 19 are visible: the weight map supplies a rate constant per synapse, the marking supplies multiplicity, and the geometric factor is 1. The dashed arrow is the refinement entry’s update function, which is where plasticity lives.

included. The unrolled network in the machine-learning sense is the reduction graph; the hidden state at a given moment is the *marking*, the standing multiset M of pending spikes on the restricted channels. Since the marking is part of the term and the term is part of the configuration, hidden state, weights and time are carried by one object.

And the threshold is in the join structure, which is the next subsection.

8.3 The keys, and why they form a partition

Definition 39 (Synaptic keys). For each synapse (j, i) put

$$\varphi_{ji} \triangleq \text{in}(a_{ji})\top \mid \text{out}(a_{ji})\top,$$

the formula holding of a communication redex on a_{ji} , and let $\Phi_{\text{COMM}} = \{\varphi_{ji}\} \cup \{\varphi_{\perp}\}$ with $\varphi_{\perp} = L^{\sharp} \wedge \neg \bigvee_{ji} \varphi_{ji}$.

Proposition 40. Φ_{COMM} is admissible, with $\alpha = 2$.

Proof. Distinct synapses are distinct channels, and a communication redex is on exactly one channel, so (P1) holds; $\varphi_{\perp}$ gives (P2) by Remark 8. Each key has depth 2 by Definition 3, and checking $\text{in}(a)\top \mid \text{out}(a)\top$ against a term is a bag lookup, so (R1) and (R2) hold. $\square$

The weight $w(\varphi_{ji})$ is the efficacy of synapse (j, i) . Nothing else in the presentation mentions it.

8.4 Threshold is join structure

Proposition 41 (McCulloch–Pitts). *The threshold presentation of Definition 38 realises the monotone Boolean threshold function $\mathbf{1}[\sum_j \mathbf{1}[a_{ji} \text{ populated}] \geq \theta_i]$: the neuron has an enabled redex exactly when at least θ_i of its dendrites carry a pending spike. It uses $\binom{k_i}{\theta_i}$ join groups.*

Proof. A group S is enabled iff every a_{si} for $s \in S$ carries a spike, since $\&$ requires simultaneous availability of all its binds; the receipt is enabled iff some group is, since $;$ separates alternatives. Some θ_i -subset is fully populated iff at least θ_i dendrites are populated. The count of groups is the count of θ_i -subsets. $\square$

So the historical origin of the neuron model — McCulloch and Pitts’ threshold unit [11] — is not encoded in a weighted GSLT but *is* a for-comprehension, with the threshold appearing as the arity of the joins and the alternatives as the group separator. Two remarks temper this.

Remark 42 (The combinatorial blowup, and the **where** clause). $\binom{k}{\theta}$ groups is unusable for realistic fan-in. Rholang 1.4's for-comprehension carries a **where** clause [16], which collapses the presentation to a single group at the cost of changing the semantics: **for**($x_1 \Leftarrow a_{1i} \ \& \ \dots \ \& \ x_k \Leftarrow a_{ki}$ **where** ϕ) waits for *all* k dendrites and then gates on ϕ . That is a coincidence detector with a filter, not a threshold unit. The two presentations agree at $\theta = k$ and differ elsewhere, and which one a modeller wants is a modelling question. We use $\theta_i = 1$ below, where both are trivial and the group presentation has k_i single-bind alternatives.

Remark 43 (Coincidence versus rate). $\theta_i = k_i$ gives a pure coincidence detector: the neuron fires only on simultaneous arrival at every dendrite. $\theta_i = 1$ gives a pure rate integrator: any arrival can trigger it, and the weights determine how fast. Real cortical neurons sit between, and so does the presentation, with θ as the dial. This is a rare case in which a biological parameter and a syntactic parameter are the same parameter.

8.5 The weighted sum, and where the squashing function went

Take $\theta_i = 1$, so N_i has k_i single-bind alternatives, and let the marking assign m_{ji} pending spikes to dendrite a_{ji} .

Proposition 44 (Propensity of a neuron). *With $g \equiv 1$ and $\chi \equiv 1$, the total propensity of the redexes belonging to neuron i is*

$$\mathbf{a}_i = \sum_j w(\varphi_{ji}) \cdot m_{ji}.$$

Proof. The receipt is persistent, so exactly one input is available on each a_{ji} , and the marking supplies m_{ji} outputs; by Definition 2 the number of redexes classified φ_{ji} is $1 \cdot m_{ji}$. Definition 19 gives $w(\varphi_{ji})m_{ji}$ per synapse, and the classes are disjoint by Proposition 40, so Proposition 21 lets them be summed. $\square$

This is exactly the weighted sum $\sum_j w_{ji}m_{ji}$ of the textbook artificial neuron, with the presynaptic activity m_{ji} read as the marking. It has not been written into any term; it is the propensity that Definition 19 computes.

Corollary 45 (The response is logistic and nobody wrote it). *Hold the marking fixed over a short window $[0, \Delta]$. The probability that neuron i fires at least once in the window is*

$$\Pr[i \text{ fires}] = 1 - \exp(-\Delta \sum_j w(\varphi_{ji})m_{ji}),$$

a saturating, monotone function of the weighted sum, with slope Δ at the origin and asymptote 1.

Proof. Firings of i within the window are the events of a Poisson process of rate $\mathbf{a}_i$ under the frozen marking, by Theorem 24 restricted to the sub-chain in which no other transition alters $m_{\cdot i}$. $\square$

The proviso is real: the marking is not in fact frozen, since other neurons are firing into it, and Corollary 45 is a statement about a window short enough that the input has not moved. That is the usual justification for a rate-coded neuron, and it is the usual approximation. What is not an approximation is the observation that no squashing function was supplied. The saturation is the exponential waiting-time law of the chain: a neuron with enormous drive cannot fire more than once per event, so its firing probability per window is bounded by 1 no matter how large the sum gets. The logistic nonlinearity of the artificial neuron is a compressed description of a Poisson process, and here it is the Poisson process.

8.6 Boundedness

Lemma 46 (Boundedness). *Let $f_i = |\text{post}(i)|$ be the fan-out of neuron i . If $f_i \leq \theta_i$ for every i , the total number of pending spikes is non-increasing along every run, the reachable term set is finite, and **Net** is term-finite in the sense of Definition 28.*

Proof. Firing N_i consumes θ_i pending spikes (one per bind in the selected group) and produces f_i . The total marking therefore changes by $f_i - \theta_i \leq 0$. Since the neurons are persistent and the channel set is fixed, a term reachable from **Net** is determined by its marking, and markings with at most $|M_0|$ tokens over finitely many channels are finite in number. $\square$

The hypothesis is restrictive: it forbids fan-out exceeding threshold, which most networks want. Two standard repairs, both expressible in the theory:

1. *Capacity.* Guard the neuron body's sends by a **where** clause bounding the occupancy of the target channel by c . The marking is then bounded by c per channel and the state space by $c^{|a|}$, restoring term-finiteness unconditionally. This is a saturating synapse, and it is what a real one does.
2. *Leak.* Add a base rule $a_{ji}!(\text{spk}) \rightsquigarrow \mathbf{0}$ with its own key and weight λ_{leak} . This does not restore term-finiteness by itself but makes the marking positive-recurrent, which is enough for the stationary analysis and for classical simulation, though not for §7.

We take capacity in what follows, since §8.10 needs a finite-dimensional $\mathcal{H}$.

Remark 47 (The network is a marked graph). Under Lemma 46, **Net** is a live, bounded, conservative marked graph in the Petri-net sense, and the analysis of [10] applies verbatim: a *period* is a firing schedule returning the marking to its initial value, and it is the right analogue of a timestep. The weighted-GSLT layer adds what a marked graph does not have, namely a probability distribution over which of the enabled transitions fires next and how long it takes.

8.7 Plasticity is the update function

Definition 48 (Hebbian entry). Equip the communication rule's entry at φ_{ji} with

$$u_{ji}(w) = w[\varphi_{ji} \mapsto \min\{w_{\max}, w(\varphi_{ji}) + \eta\}],$$

for a potentiation increment $\eta > 0$ and ceiling $w_{\max}$.

So the augmented rule reads, in full:

```

COMM . |- (PPar {(PForUser rows p), (POutput n q), ...rest})
      ~> (PPar {(subst ...), ...rest}) [1.0] {
    phi(0,1) => (w_01, hebb(0,1)),
    phi(1,1) => (w_11, hebb(1,1)),
    ...
    phi_bot => (0.0, id)
  };
ParCong . | S ~> T |- (PPar {S, ...rest}) ~> (PPar {T, ...rest}) [1.0] { snd };

```

Theorem 49 (Learning and inference are one relation). *Let **Net** be a weighted GSLT as above. The transition relation of Definition 16 on $\mathcal{C} = \mathcal{T} \times \mathcal{W}$ has two marginals: the projection to $\mathcal{T}$ is the network's inference dynamics, and the projection to $\mathcal{W}$ is its learning dynamics. Neither is a separate process. In particular there is no training schedule, no separate learning clock, and no distinguished learning phase; the learning rate η is not a hyperparameter of an outer loop but a component of the theory, sitting in the same refinement entry as the weight it modifies.*

Proof. Immediate from Definitions 13 and 16: a single step produces (P', w') with P' obtained by the rewrite and w' by the fold of the update function. $\square$

This is the claim the example exists to make, and it is exactly what a static-rate calculus cannot express. In SPiM a rate is fixed at declaration, so a stochastic π model of a neural network can model the firing but not the learning; one must step outside the calculus, adjust the rates, and re-run. Here the adjustment is a transition of the same system.

Remark 50 (Cells that fire together wire together, literally). Definition 48 potentiates φ_{ji} when a communication on a_{ji} fires — that is, when a presynaptic spike is consumed by a postsynaptic receipt. The condition “pre and post were jointly involved in an event” is not a correlation computed by an observer; it is the firing of a single rewrite, which by construction requires both parties. Hebb’s rule is the statement that the event which transmits is the event which potentiates [9], and here those are the same event because a communication is one rewrite.

8.8 Spike-timing dependence needs a wider update signature

Spike-timing-dependent plasticity makes Δw a function of the interval between pre- and postsynaptic spikes, potentiating for pre-before-post and depressing for the reverse [21]. The ingredients are available — the simulator of Definition 23 produces a timestamp at every step — but the update function of Definition 14 cannot see them, since its type is $\mathcal{W} \rightarrow \mathcal{W}$.

Definition 51 (Extended update). Replace $u_i : \mathcal{W} \rightarrow \mathcal{W}$ by

$$u_i : \mathcal{W} \times \text{Match} \times \partial\mathcal{T} \times \mathbb{R}_{\geq 0} \times \mathcal{S} \longrightarrow \mathcal{W},$$

taking additionally the matched substitution σ , the position k at which the rule fired, the current simulation time t , and a bounded *trace summary* $\mathcal{S}$ — for STDP, one eligibility trace per synapse, itself decaying at a rate specified in the theory.

Theorem 24 survives this extension provided $\mathcal{S}$ is finite-dimensional and updated deterministically, since one simply enlarges the configuration to $(P, w, \mathcal{S})$; it does *not* survive an update function reading unbounded history, which would make the process genuinely non-Markovian and put exact simulation out of reach. The eligibility-trace formulation of STDP is precisely the standard device for staying inside the bounded case, and it is pleasant that the constraint arrives here from the requirement that the chain be a chain rather than from computational convenience.

8.9 Uniformity: one theory for every network

Definition 39 gives one key per synapse, so a network with 10^4 synapses has 10^4 keys and a network that grows needs a new theory. Reflection removes both problems. Following the construction of [10], take the synapse channels to be structured names

$$a_{ji} = @[*net, j, i],$$

so that a name predicate can recover the endpoints. Then

$$\varphi_{\text{syn}} \triangleq \exists j, i. \text{in}(@[*net, j, i])\top \mid \text{out}(@[*net, j, i])\top$$

is a single key describing the whole *namespace* of synapses of this network, which is namespace logic doing what it was built for [15]. The weight map then has one entry per network rather than per synapse; per-synapse efficacies are recovered by letting the entry’s value be a function

of the recovered indices, which is the standard move of trading a large map for a small map with structured keys. A greatest fixed point makes “every neuron reachable from i eventually fires” a single formula rather than an n -fold conjunction, which is the uniformity ν buys [1, §3]. In an atomic-name calculus none of this is available: the indices would have to travel as message payloads and the logic could reach them only by observing traffic.

8.10 The quantum network

Replace the efficacies by amplitudes $z_{ji} \in \mathbb{C}$, $|z_{ji}| \leq 1$, keeping the keys, the neurons, and the capacity bound. Definition 30 gives a jump operator per synapse, Definition 32 a Lindbladian, and Definition 34 a trajectory simulator. Because the capacity bound makes $\mathcal{B}$ finite, $\mathcal{H}$ is finite-dimensional and the model checking of [22, 23] applies, with the projectors supplied by Proposition 36 — for instance $\Pi_{\varphi_{\text{syn}}}$, whose expectation is the probability that the network has an available synaptic transmission at time t .

Remark 52 (Indistinguishable spikes interfere). By Remark 22, m indistinguishable pending spikes on a_{ji} give m redexes with a common contractum. Classically their rates add and the transition runs at $m \cdot w(\varphi_{ji})$. In the complex case their *amplitudes* add, so the amplitude to the target is $m z_{ji}$ and the probability weight is $m^2 |z_{ji}|^2$. The quantum network is therefore not the classical one with phases attached: it is superlinear in presynaptic multiplicity where the classical one is linear, and the enhancement is exactly a consequence of the spikes being elements of a bag rather than a list. Whether that is a feature or an artefact of the encoding is a genuine question — one could break the degeneracy by tagging spikes, recovering linearity — and we flag it rather than settle it.

8.11 What this example does not show

Honesty about scope. The learning here is local and Hebbian; nothing above implements backpropagation, which needs a global error signal propagated against the direction of inference. One can encode such a signal as traffic on a second family of channels with its own keys, and the apparatus does not forbid it, but the result would no longer have the property that makes Theorem 49 interesting — the error channel is an outer loop wearing a costume. Whether a genuinely local weighted-GSLT learning rule can match backpropagation is not a question this note answers, and the honest statement is that the example demonstrates a *mechanism* for plasticity inside the calculus, not a competitive learning algorithm.

Nor do we claim biological fidelity. The neuron of Definition 38 has no membrane potential, no refractory period, and no dendritic geometry. The first two are easy additions — a potential is a payload, a refractory period is a guard — and the third is what the geometric factor g of §4.3 is for, which is a use we have not developed.

9 Implementation

9.1 Surface syntax

The weighting is a decoration of MeTTaIL’s **rewrites** block, and it should read as one. The existing surface is

```
rewrites {
  Exec . |- (PDrop (NQuote P)) ~> P;
  ParCong . | S ~> T |- (PPar {S, ...rest}) ~> (PPar {T, ...rest});
}
```

and the weighted surface adds a bracketed weight and a brace block after the rule, in both cases optional:

```
rewrites {
  // base rule: [scalar] { key => (weight, update) }
  COMM . |- (PPar {(PForUser rows p), (POutput n q), ...rest})
    ~> (PPar {(subst rows p q), ...rest}) [1.0] {
    in(n)TOP | out(n)@fast => (2.5, scale(0.9)),
    in(n)TOP | out(n)TOP => (0.7, id),
    default => (0.0, id)
  };

  // base rule, unweighted: defaults to [1.0] { default => (1.0, id) }
  Exec . |- (PDrop (NQuote P)) ~> P;

  // context rule: [geometric factor] { fold }
  ParCong . | S ~> T |- (PPar {S, ...rest}) ~> (PPar {T, ...rest}) [1.0] { snd
  };
}
```

Notes on the design. **default** is sugar for the completion of Remark 8 — the elaborator synthesises $L^\sharp \wedge \neg \bigvee_i \varphi_i$ — so a modeller writes exclusive keys and gets a partition. Exclusivity itself is checked at elaboration time and is a hard error, not a warning, since Proposition 7 fails without it. Keys are written in the concrete syntax of the generated logic; the two shown are ordered from more to less specific purely for readability, since exclusivity means order is semantically irrelevant. A context rule’s default factor is 1 and its default fold is **snd**, which together implement “addressing is free and does not touch the map” (§4.3).

9.2 Delta against the prototype

The prototype at `rho-life/Gillespie` [8] implements a substantial part of this. The following is what separates it from the specification above, in descending order of importance.

1. *Propensity ignores the term.* `BaseRule::propensity()` returns the rule weight times the sum of its refinement weights, with no dependence on the configuration. Gillespie’s multiplicity factor — the cardinality of $K(r, \varphi, P)$ in Definition 19 — is absent, so a term with fifty available redexes of a kind fires them no faster than a term with one. This is the one correctness bug rather than a design divergence, and it requires a matcher over the term, which the crate does not currently have (its `TermRef` is an opaque id, sort and display string).
2. *Spatial behaviours are hand-rolled.* The `SpatialBehavior` enum (`Local`, `Interaction`, `Parallel`, `Sequential`, `Guard`, `Replicated`, `Custom`) is a fixed vocabulary written by hand. Keys must instead be formulae of the logic *generated* from the language spec, or the construction loses the property that makes it generic over GSLTs. This is the largest gap between crate and specification and the one gated on the branch’s logic work.
3. *No partition check.* Nothing enforces (P1) or (P2), and `RateMap::select_classical` picks by cumulative weight over the entries as declared, which silently implements the “sum over all satisfied keys” aggregation. Elaboration-time exclusivity checking and **default** synthesis are required.
4. *Context rules carry independent propensity.* `ContextRule` has its own weight and enters the propensity sum alongside base rules, which double-counts (§4.3). It should contribute $g(k)$ multiplicatively to the base firing it addresses.

5. *Rates are clamped to $[0, 1]$.* `RateValue::real` rejects values outside the unit interval and `RateValue::add` clamps sums to 1. Per Remark 11 the codomain is $\mathbb{R}_{\geq 0}$; the clamping additionally makes `a0` wrong whenever two classes are simultaneously enabled with rates summing above 1.
6. *Update functions are too narrow.* `UpdateFn = Fn(&RateMap) -> RateMap` cannot see the match, the position, or the clock, so Definition 48 is expressible but §8.8 is not. Widen to Definition 51.
7. *The quantum module is not a QCTMC.* `quantum.rs` maintains a superposition of `(TermRef, Complex64, RateMap)` triples and measures by $|z|^2$. That is amplitude bookkeeping over a classical branching structure; reaching [22] needs density operators, the jump operators of Definition 30, the Lindbladian of Definition 32, and the non-Hermitian evolution of Definition 34. In particular there is currently no H_{eff} , so the waiting time cannot be drawn from a decaying norm and Theorem 35’s general case is unreachable.
8. *Syntax drift.* `language_ext.rs` documents context rules as `if <cond> then <lhs> ~> <rhs>`, where MeTTaIL writes `| S ~> T |- ....`. Also, `lib.rs` declares a `fuzzer` module that is not present in the tree.

Items 1, 3, 4, 5 and 8 are local and can be done now. Item 6 is a signature change with a configuration-space consequence (§8.8). Item 2 is gated on the generated logic, which is still under development in the feature branch; until it lands, the honest interim is to keep a hand-rolled key vocabulary but to make it an *adapter* behind the same interface the generated formulae will implement, so the swap is at the boundary rather than through the simulator. Item 7 is a rewrite of one module.

9.3 Incremental propensity

Lemma 10 licenses the standard optimisation. Maintain `a` as a Fenwick tree over (r, φ) classes with per-class redex lists; after firing at position k , reclassify only redexes within radius $\alpha + |L| + |R|$ of k , and additionally rescale the classes whose weight the update function touched — which for Definition 48 is exactly one class. The second part is the only respect in which dynamic weight maps cost anything at run time, and it is cheap precisely when update functions are sparse. A modeller who writes an update function touching every entry pays a full recomputation of `a0` per step, and should be told so.

10 Weights and costs

Cost accounting and weighting both decorate rewrites, and it is worth stating plainly that they are different decorations doing different jobs.

A cost decoration attaches to a rewrite a debit against a token stack, and the funding judgment $\Sigma \vdash_{\S} \Delta$ — decidable, total, monotone in supply — decides whether a step may occur at all [2]. It is implemented in the transpiler branch as `OsIfResourceLogic`, with a supply multiset keyed by signature and a demand list, one token per signed term, and conformance laws asserting soundness, rejection of underfunding, supply monotonicity, and decidability. A weight decoration attaches a rate, and decides how fast and how often. The composition is one-directional:

Definition 53 (Funding gate). For a redex (k, σ) of rule r , let $\Delta(r, k, \sigma)$ be the demand it incurs and $\Sigma(P, k)$ the supply available at that location. Put $\chi(r, k, \sigma) = 1$ if $\Sigma(P, k) \vdash_{\S} \Delta(r, k, \sigma)$ and 0 otherwise.

Proposition 54 (Cost gates the support, weight distributes over it). *With χ as in Definition 53, the coalgebra of Definition 17 restricts to the funded sub-transition-system: unfunded redexes have propensity zero and contribute nothing to $\mathbf{a}_0$, while among funded redexes the distribution is unchanged from the cost-free case up to normalisation.*

Proof. χ appears as a multiplicative factor in Definition 19, taking values in $\{0, 1\}$; a zero factor removes the term from the sum, and the remaining terms are untouched. $\square$

Three consequences worth recording.

The gate is decidable, so the simulator stays exact. If funding were a real-valued penalty rather than a Boolean, the propensity would depend on an optimisation and the chain would no longer be simulable by Definition 23. The decidability law of the resource logic is doing real work here.

Exhaustion is a halting condition, not a deadlock. $\mathbf{a}_0 = 0$ when every enabled redex is unfunded, and Definition 23 halts. The chain has reached an absorbing state, which is the correct reading: a system that cannot pay for any of its available transitions has stopped, and the waiting time to the next event is infinite rather than undefined.

The neural example acquires a metabolism. Give the communication rule a phlogiston demand per firing and the network of §8 has an energy budget: a neuron whose local supply is exhausted has propensity zero regardless of its synaptic weights, and the network's activity is bounded by its funding rather than by its capacity guard. Weight is what a synapse is worth; cost is what a spike costs. Separating them is what lets a model say that a strong synapse on a starving neuron is silent, which is a statement neither decoration makes alone. This is the point of contact with the mortal-computation reading of [18], and we go no further with it here.

11 Open threads

1. *A combined logic.* §7.5 has the generated logic supplying projectors to an external temporal logic. What one wants is a single logic with context-labelled modalities and time-bounded operators — $\langle K_j \rangle^{\leq t} \varphi$ — adequate for weighted bisimulation on configurations. The classical half of this is well trodden (CSL over CTMCs); the question is what OSLF emits when the presentation it is fed is already weighted.
2. *Weighted bisimulation.* The category **GSLT** has bisimulation-preserving maps as morphisms [16]. What is the right notion when the objects carry weight maps? Rate bisimulation (lumpability) is the obvious candidate for $\mathbb{R}_{\geq 0}$; the $\mathbb{C}$ case has no obvious candidate, and the interference of Proposition 33 suggests it should not be a relation on terms alone.
3. *Geometric factors.* §4.3 identifies g as the natural home of a spatial weighting and declines to develop it. The gravitation correspondence of [3] splits a weight on a reduction-graph edge into a geometric part and a matter part; here $g(k)$ is visibly the first and $w(\varphi)$ visibly the second, and the correspondence asks for an action forcing the two to agree. That is a paper.
4. *Quantale weights.* A quantale-valued codomain gives a fuzzy calculus, and [5] develops the flat-substitution instance independently. Whether it is an instance of Definition 12 or a genuinely different decoration — the fuzzy comm rule transmits a modified payload, which no weighting here does — is not settled, and settling it is the natural way to test whether $\mathcal{K}$ is really a parameter.

5. *Map-finiteness*. §6.3 notes that plastic weights leave the reachable map set infinite unless quantised or saturated. Quantised weights make $\mathcal{C}$ finite and the whole apparatus model-checkable; they also make the learning dynamics a finite automaton, whose reachable weight configurations are an object nobody has looked at.
6. *The bosonic enhancement*. Remark 52 observes that indistinguishable messages give amplitude mz rather than rate $m|z|^2$, so the quantum reading is superlinear in multiplicity. Whether this is a feature of the physics being modelled or an artefact of the associative-commutative presentation is open, and it is the sharpest available test of whether the complex case is a genuine quantum semantics or a formal analogy.

References

- [1] L. G. Meredith. *Classifying knots with spatial-behavioural logics*. F1R3FLY.io working note, 2026.
- [2] L. G. Meredith. *Cost accounting as an endofunctor on continued interactive GSLTs*. F1R3FLY.io working note, 2026.
- [3] L. G. Meredith. *Cost, spacetime, and the geometry/matter split on reduction graphs*. F1R3FLY.io working notes, 2026.
- [4] J. Dalibard, Y. Castin and K. Mølmer. Wave-function approach to dissipative processes in quantum optics. *Physical Review Letters* 68(5):580–583, 1992.
- [5] *Fuzzy rho-calculus: a quantale-graded interaction, and its lift to ciGSLTs with binding*. F1R3FLY.io working note, 2026.
- [6] M. A. Gibson and J. Bruck. Efficient exact stochastic simulation of chemical systems with many species and many channels. *Journal of Physical Chemistry A* 104(9):1876–1889, 2000.
- [7] D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *Journal of Physical Chemistry* 81(25):2340–2361, 1977.
- [8] F1R3FLY.io. `mettail-gillespie`: stochastic and quantum Gillespie simulation extensions for MeTTaIL rewrite rules. `F1R3FLY-io/publications, rho-life/Gillespie`, 2026.
- [9] D. O. Hebb. *The Organization of Behavior*. Wiley, 1949.
- [10] L. G. Meredith. *Knots as cost-accounted processes*. F1R3FLY.io working note, 2026.
- [11] W. S. McCulloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics* 5:115–133, 1943.
- [12] K. Mølmer, Y. Castin and J. Dalibard. Monte Carlo wave-function method in quantum optics. *Journal of the Optical Society of America B* 10(3):524–538, 1993.
- [13] L. G. Meredith et al. *The MeTTa calculus*. F1R3FLY.io, 2026.
- [14] F1R3FLY.io. `mettail-rust`: the `language!` macro and its shipped language definitions. `F1R3FLY-io/mettail-rust`, 2026.
- [15] L. G. Meredith and M. Radestock. Namespace logic: a logic for a reflective higher-order calculus. In *Trustworthy Global Computing*, LNCS 3705, Springer, 2005.

- [16] L. G. Meredith. *Graph-Structured Lambda Theories: A Reference for the Working Developer*. F1R3FLY.io, 2026.
- [17] C. Priami. Stochastic π -calculus. *The Computer Journal* 38(7):578–589, 1995.
- [18] L. G. Meredith. *The Mortal Scientist*. F1R3FLY.io working note, 2026.
- [19] A. Phillips and L. Cardelli. A correct abstract machine for the stochastic pi-calculus. In *Bioconcur*, 2004.
- [20] A. Phillips and L. Cardelli. Efficient, correct simulation of biological processes in the stochastic pi-calculus. In *Computational Methods in Systems Biology*, LNCS 4695, Springer, 2007.
- [21] G. Q. Bi and M. M. Poo. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience* 18(24):10464–10472, 1998.
- [22] M. Xu, J. Mei, J. Guan and N. Yu. Model checking quantum continuous-time Markov chains. In *CONCUR 2021*, LIPIcs 203, 13:1–13:17. Schloss Dagstuhl, 2021. Preprint [arXiv:2105.00382](#).
- [23] M. Xu, J. Mei, J. Guan and N. Yu. Checking continuous stochastic logic against quantum continuous-time Markov chains. *Logical Methods in Computer Science*, 2025. Preprint [arXiv:2202.05412](#).